Лабораторная работа № 2 — Статический UI приложения: экраны темы в Material 3

Дисциплина	Программирование мобильных приложений (PAM)
Период	недели 4–5
Защита	индивидуальная, на занятиях
Автор	унив. лектор Ionel Antohi

1. Цели работы

По завершении работы студент способен:

1. создать Flutter-проект с ясной структурой директорий (экраны в `lib/screens/`, переиспользуемые компоненты в `lib/widgets/`) и работать инкрементально, с Git-commit-ами по ходу работы;
2. задать визуальную тему приложения с помощью `ThemeData` и `ColorScheme.fromSeed`, соответствующую домену темы проекта;
3. построить в виде статического UI с зашитыми в код данными **все обязательные экраны** из спецификации темы, выбранной из каталога;
4. компоновать layout-ы Material 3 из фундаментальных widget-ов (`Scaffold`, `AppBar`, `Column`/`Row`, `ListView`, `Card`, `TextField`, кнопки);
5. извлекать переиспользуемые widget-ы из кода, повторяющегося между экранами;
6. запустить и продемонстрировать приложение на эмуляторе или на физическом телефоне.

Чего эта работа НЕ делает (чтобы вы не трудились наперёд впустую): реальная навигация с `go_router`, валидация форм и любая логика аутентификации — это **материал L3**; состояние приложения приходит на L4, а backend — на L5. На L2 приложение — **функционально-визуальный макет**: выглядит в точности как конечный продукт, но данные зашиты в код, а кнопки могут оставаться без эффекта.

2. Предварительные условия

- **L1 защищена**, тема проекта **выбрана из каталога и зафиксирована** преподавателем (или утверждена собственная тема). Напоминаем правило из каталога: одна смена темы ещё возможна, с согласия преподавателя, **до защиты L2** — после этого тема окончательна.
- **Лекции C4–C5 пройдены**: widget-ы, `StatelessWidget` / `StatefulWidget`, дерево widget-ов (C4); `Row`/`Column`/`Stack`, ограничения layout-а, Material 3, темы (C5).
- Среда из L1 работоспособна: чистый `flutter doctor`, доступен эмулятор или физический телефон.

Держите открытой **спецификацию вашей темы** из каталога (далее также «ТЗ темы» — описание, обязательные экраны и модель данных): на L2 статически строятся **все экраны из раздела *обязательные экраны***; пункт «L2» темы лишь выделяет некоторые из них, а не сужает список.

3. Задание 1 — Проект приложения и дисциплина Git

Создайте проект с именем темы (строчные буквы, слова через подчёркивание):

```
flutter create medi_shop        # пример: тема T1; используйте имя ВАШЕЙ темы
cd medi_shop
```

Организуйте исходники с самого начала — на L3–L6 проект только растёт, а не реорганизуется:

```
lib/
├── main.dart          # MaterialApp + тема + стартовый экран
├── screens/           # по одному файлу на экран: login_screen.dart, ...
├── widgets/           # переиспользуемые компоненты: item_card.dart, ...
└── data/              # (опционально) mock_data.dart — зашитые в код данные,
                       #  используемые несколькими экранами
```

Mock-данные, определённые и используемые в одном-единственном экране, могут находиться в файле этого экрана; то, что используется на нескольких экранах, переносите в `lib/data/mock_data.dart`, чтобы не копировать один и тот же список дважды.

Инициализируйте Git-репозиторий и работайте **инкрементально** — commit после каждого завершённого шага (проект создан, визуальная тема применена, каждый экран закончен):

```
git init
git add .
git commit -m "Начальный проект: flutter create"
# ... после каждого экрана:
git add .
git commit -m "Экран каталога: статический список товаров"
```

`flutter create` уже генерирует корректный `.gitignore` — не удаляйте его. История `git log --oneline` показывается на защите: она доказывает, что приложение росло шаг за шагом, а не появилось за один вечер. Минимальное требование: **не менее одного commit-а на экран**, с сообщениями, которые говорят, что было добавлено.

4. Задание 2 — Визуальная тема Material 3

Приложение **не остаётся на фиолетовой теме по умолчанию** из сгенерированного шаблона. Выберите seed-цвет, соответствующий домену темы (зелёный для приложения о

садоводстве, медицинский синий для аптеки, оранжевый для еды…), и позвольте `ColorScheme.fromSeed` вывести из него всю палитру — ровно как на C5:

```dart
// main.dart
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Vinyl Shelf', // название ВАШЕГО приложения
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6D4C41), // цвет вашего домена
        ),
        // базовая типографика: точечные подстройки, остальное по умолчанию
        textTheme: const TextTheme(
          titleLarge: TextStyle(fontWeight: FontWeight.w600),
        ),
      ),
      home: const HomeScreen(),
    );
  }
}
```

В экранах **не зашивайте цвета в код**: используйте `Theme.of(context).colorScheme.primary`, `...surfaceContainerHighest` и т.д. — так весь UI перекрашивается из одного места, а на защите вы можете продемонстрировать это, поменяв `seedColor` и выполнив hot reload.

5. Задание 3 — Все обязательные экраны как статический UI

Постройте **каждый экран из раздела «обязательные экраны»** вашей темы (6–8 экранов), каждый в **своём файле** в `lib/screens/`. Правила:

1. **Данные зашиты в код**: списки mock-объектов, определённые локально, — без сети, без баз данных, без `setState` на содержимом.
2. **Содержимое реалистично для домена темы**: в аптеке есть «Парацетамол 500 мг — 18,50 MDL», а не «Item 1» и не «Lorem ipsum». **Не менее 6 элементов** (рекомендуется 8) в каждом списке, с полями из модели данных темы. Именно реалистичные данные вскрывают проблемы layout-а: длинное название, которое не помещается, цена с многими цифрами — поэтому мы требуем их сейчас, а не на L5.
3. **Верность ТЗ** (спецификации темы из каталога — описание, обязательные экраны, модель данных): то, что обещает спецификация экрана, должно быть видно — если

написано «поиск + фильтр по категории», на экране есть поле поиска и ряд фильтров (как визуальные элементы; работать они начнут только на L4). На защите экраны сравниваются один за другим со списком из каталога.

Ниже — **типовой шаблон** экрана-списка на нейтральном домене, которого **нет в каталоге** (личная коллекция виниловых пластинок), — именно затем, чтобы его нельзя было скопировать: вы его **адаптируете** под сущности вашей темы.

```dart
// data/mock_data.dart — mock-модель: поля берутся из модели темы
class VinylRecord {
  final String title;
  final String artist;
  final String genre;
  final double price;
  const VinylRecord(this.title, this.artist, this.genre, this.price);
}

// зашитые в код данные, реалистичные для домена
const mockRecords = [
  VinylRecord('Kind of Blue', 'Miles Davis', 'jazz', 350),
  VinylRecord('Abbey Road', 'The Beatles', 'rock', 420),
  VinylRecord('Discovery', 'Daft Punk', 'electro', 380),
  // ... не менее 6 элементов
];
```

```dart
// screens/collection_screen.dart
import 'package:flutter/material.dart';
import '../data/mock_data.dart';
import '../widgets/record_card.dart';

class CollectionScreen extends StatelessWidget {
  const CollectionScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Моя коллекция')),
      body: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: mockRecords.length,
        separatorBuilder: (_, __) => const SizedBox(height: 8),
        itemBuilder: (context, index) =>
            RecordCard(record: mockRecords[index]),
      ),
    );
  }
}
```

Обратите внимание на шаблоны, которые стоит запомнить: `ListView.builder`/`.separated` (а не `Column` с 8 дочерними элементами, выписанными вручную), извлечённый widget-карточка (Задание 5) и `StatelessWidget` повсюду — на L2 ни один экран ещё не имеет собственного состояния.

Для экрана **деталей** используйте тот же принцип: mock-объект, отображаемый через `Column` + `Row`, изображение или `Icon`-заглушка, `Card`/`ListTile` для групп полей. Экраны типа **форма** («новая сессия», «оформление заказа») строятся из `TextField` / `DropdownMenu` / `Switch` **без какой-либо валидации и без сохранения** — это макеты, ровно как экран аутентификации ниже. Экраны с **изображениями** могут на этом этапе использовать крупные `Icon`-ы, `Image.asset` с 2–3 локальными изображениями (объявленными в `assets:` в `pubspec.yaml`, как на С5) или цветной `Container` с инициалами — свой выбор задокументируйте в отчёте.

6. Задание 4 — Экран аутентификации: строго статический

У каждой темы в каталоге есть экран *Вход / регистрация*. На L2 он **только визуальный**, согласно правилу из каталога: статический сейчас, **валидация приходит на L3, подключение к backend-у — на L5**. Конкретно:

- два `TextField`-а (e-mail/пользователь и пароль, с `obscureText: true`) и кнопка `FilledButton` «Войти»;
- опционально — ссылка «Нет аккаунта? Зарегистрируйтесь» как простой `TextButton`;
- **без** `Form`, **без** `validator:`, **без** какой-либо проверки полей, **без** логики по нажатию кнопки (максимум — переход на стартовый экран, чтобы не блокировать демонстрацию).

Кто «успеет» написать валидацию сейчас, всё равно перепишет её на L3 — на `Form`/`TextFormField`, как преподаётся на С6, — не забегайте вперёд.

7. Задание 5 — Переиспользуемые widget-ы

Извлеките **не менее двух** переиспользуемых widget-ов в `lib/widgets/`. Классический кандидат — карточка элемента списка, используемая на 2–3 экранах (каталог, избранное, история…). Вторым может быть всё, что повторяется: бейдж статуса заказа, строка «метка: значение» на экране деталей, аватар с инициалами.

```dart
// widgets/record_card.dart
import 'package:flutter/material.dart';
import '../data/mock_data.dart';

class RecordCard extends StatelessWidget {
  final VinylRecord record;
  const RecordCard({super.key, required this.record});

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;
    return Card(
      child: ListTile(
        leading: CircleAvatar(
          backgroundColor: scheme.primaryContainer,
          child: Text(record.title[0]), // инициал вместо обложки
        ),
        title: Text(record.title),
        subtitle: Text('${record.artist} · ${record.genre}'),
        trailing: Text(
          '${record.price.toStringAsFixed(0)} MDL',
          style: Theme.of(context).textTheme.titleMedium,
        ),
      ),
    );
  }
}
```

Критерий успеха: если вы меняете вид карточки в одном-единственном файле, она меняется на всех экранах, которые её используют. Параметризуйте через конструктор (отображаемый объект, при необходимости `onTap`), а не через copy-paste.

8. Задание 6 — Минимальная навигация (только для демонстрации)

Реальная навигация — именованные маршруты, параметры, `go_router` — это **материал L3**; на L2 вы её не строите. Вам нужно лишь одно: **на защите иметь возможность добраться до каждого экрана**. Принимаются два варианта, выберите один:

1. **Стартовый экран с кнопками** — список `FilledButton`/`ListTile`, по одной кнопке на экран, с простым `Navigator.push`:

   ```dart
   FilledButton(
     onPressed: () => Navigator.push(
       context,
       MaterialPageRoute(builder: (_) => const CollectionScreen()),
     ),
     child: const Text('Коллекция'),
   ),
   ```

2. **Простой `NavigationBar` (Material 3)** — 3–5 основных пунктов назначения, переключаемых по индексу, плюс кнопки к второстепенным экранам. Обёртка с `NavigationBar` — единственное допустимое исключение из правила «`StatelessWidget` повсюду»: минимальный `StatefulWidget`, который хранит только индекс выбранного пункта (`setState` на индексе, а не на содержимом); переключаемые экраны остаются `StatelessWidget`.

Оба варианта — временные строительные леса: на L3 их заменит реальная навигация. Не устанавливайте `go_router` сейчас; ничего сверх минимума выше не оценивается — усилия вкладываются в экраны.

9. Требования к отчёту и защита

Отчёт сдаётся в формате **PDF** с именем файла `PAM_L2_ИмяСтудента.pdf` и содержит:

1. **Стандартный титульный лист** (как на L1) + **тема проекта**: номер и название из каталога;
2. **Структура проекта**: дерево директории `lib/` (текстом) и по фразе о том, что содержит каждая директория;
3. **Визуальная тема**: фрагмент `ThemeData`/`ColorScheme.fromSeed` и обоснование выбранного цвета (1–2 фразы);
4. **Экраны**: для каждого обязательного экрана из ТЗ — скриншот с эмулятора/телефона и 1–2 фразы (что он показывает, из каких основных widget-ов составлен). Порядок и названия следуют списку из каталога;
5. **Переиспользуемые widget-ы**: для каждого — исходный код и список экранов, которые его используют;
6. **История Git**: вывод `git log --oneline` (полностью).

Защита **индивидуальная**: вы запускаете приложение на эмуляторе или на физическом телефоне, проходите экраны один за другим со спецификацией темы рядом, объясняете любой фрагмент кода по требованию и отвечаете на контрольные вопросы. Экран, код которого студент не может объяснить, не засчитывается. Оценка за L2 входит в среднюю по лабораторным, которая учитывается в двух компонентах семестровой оценки (текущее оценивание и индивидуальная работа) — как, впрочем, и каждая лабораторная проекта.

10. Критерии оценивания

Компонент	Баллы
Организованный проект (`lib/screens/`, `lib/widgets/`) + инкрементальная история Git (≥ 1 commit на экран)	1 б

Компонент	Баллы
Применена тема Material 3: `ColorScheme.fromSeed` на цвете домена, без зашитых в код цветов в экранах	1 б
Все обязательные экраны из ТЗ присутствуют и верны спецификации (по каждому экрану, пропорционально)	4 б
Качество UI и реалистичность mock-данных (layout без переполнений, не менее 6 реалистичных элементов на список)	1 б
Не менее 2 переиспользуемых widget-ов, параметризованных и используемых в нескольких местах (на нескольких экранах или повторно в одном)	1 б
Статический экран аутентификации (поля + кнопка, без валидации/логики)	1 б
Защита: приложение работает на устройстве, код объяснён, ответы на вопросы	1 б
Итого	**10 б**

Экраны, отсутствующие или не соответствующие ТЗ, пропорционально снижают компонент в 4 б; экран, «наполненный» Lorem ipsum или «Item 1/2/3», считается незавершённым.

Примеры контрольных вопросов:

1. Чем `StatelessWidget` отличается от `StatefulWidget` и почему на L2 все экраны могут оставаться `StatelessWidget`? Что изменилось бы, если бы мы хотели, чтобы работал фильтр?
2. Что означают жёлто-чёрные полосы, которые Flutter показывает у края `Row`, и как это чинится с помощью `Expanded`/`Flexible`?
3. Как цвет из `ColorScheme.fromSeed` доходит до `Card` на одном из экранов? Какую роль играет `Theme.of(context)` и почему мы не зашиваем цвета в код?
4. Почему для списков мы используем `ListView.builder`/`.separated`, а не `Column` со всеми элементами, созданными заранее?
5. Что даёт `Scaffold` отдельному экрану и что даёт `MaterialApp` всему приложению? Что теряется, если их нет?

6. Покажите в вашем приложении переиспользуемый widget: какие у него параметры и на каких экранах он появляется? Чем он лучше дублированного кода?